

seo-advisor: one page first

What we built, the new single-page plan, and every part of the code, explained in Simplified Technical English.

Subject	raksha-gurzu/seo-advisor at commit 877c666, plus the uncommitted work of 8 October
Covers	Steps 0.1 to 0.3, slices 1 and 2a, the API, and the single-page plan of 8 October
Audience	The owner: basics first, then the deep parts
Language	ASD-STE100 (Simplified Technical English)
Replaces	2026-10-08-seo-advisor-progress.pdf (version 1, before the single-page plan)
Source of truth	The code, tests, notes and git history on 8 October 2026. Every number was counted.

Contents

How to read this	4
Boxes in the text	4
Part 1 What we build	5
The page loop	5
Later: the whole site	5
Rules that never change (CLAUDE.md)	5
Part 2 Where we are	6
The numbers	6
Status of each step	6
How we work	6
Two changes of direction	7
Part 3 Words you need	8
Part 4 How the code is organised	9
The five import contracts	9
The file map	9
Part 5 The main trace: inventory:discover on MyPipit	11
The same run through the API	12
Steps and their files	12
Part 6 The safe fetcher	13
What each gate does	13
The constants	13
Where the code came from	14
Two security reviews	14
The request trace	14
The third review	15
Part 7 The inventory	16
What the experiment found	16
The MyPipit page types	16
Rules for sitemaps	16
Problems are data	16
Part 8 The database	18
The local database	18
Design points	18
The first migration	18
How the tests use the database	18
The next tables	19
Part 9 Tools and checks	20

mise: one place for versions and commands	20
What mise run check runs	20
The 7-day delay	20
Packages and licences	20
Settings	21
Part 10 Rules and decisions	22
Rules that you added to CLAUDE.md	22
Decisions that you made	22
Decisions that Claude made for you	22
Git	22
Part 11 Open items and next steps	23
Open items	23
Next steps	23
How to start the next session	23
When something fails	23

How to read this

Part 1, Part 2 and Part 11 are the spine. With 20 minutes, read only these three parts.

If you want to know	Read
What the product does (the page loop)	Part 1
What is done and what changed	Part 2
A word that you do not know	Part 3
Where the code is	Part 4
How one command works, step by step	Part 5
Why the internet path is safe	Part 6
How pages are found and typed	Part 7
How data is stored	Part 8
What each tool and check does	Part 9
Which rules and decisions apply	Part 10
What is open, and what comes next	Part 11

Boxes in the text

	Meaning
Rule	A decision that other parts depend on. A change affects several files.
Trap	Something that failed quietly, or that can fail quietly.
Note	The reason for a choice that is not obvious.

File paths without a folder start at `apps/api/src/seo_advisor/`. Commands run in the project folder.

PART 1

What we build

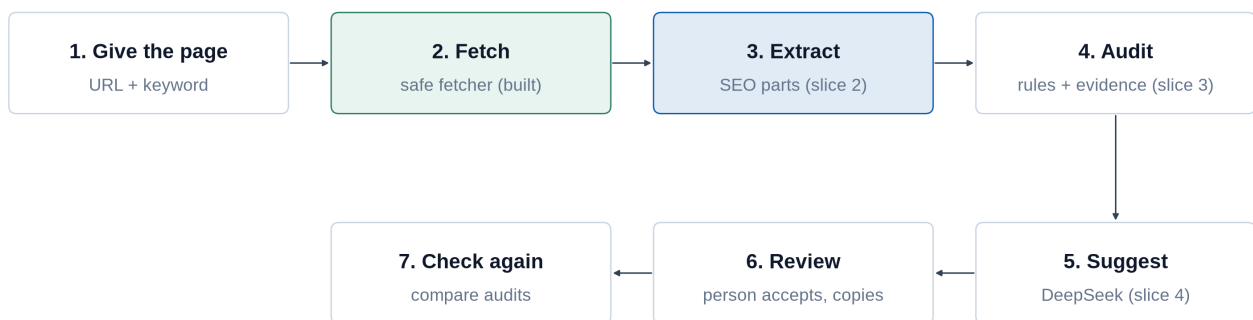
One page first. The whole site later.

seo-advisor is a tool for the Gurzu SEO team. It improves how the pages of Gurzu clients appear in Google search. On 8 October 2026 you decided to start with one page: the tool fetches one page and does SEO on that page. People make every final decision.

The page loop

One page goes through seven steps (Figure 1). The first page type is the MyPipit blog post.

The page loop: one page, seven steps. The safe fetch is built; slice 2 is next.



Green: built. Blue: next slice. White: planned in Phase 1. Whole site: Phase 3.

A person approves and saves every change. The tool never publishes.

Figure 1. The page loop. Green: built. Blue: the next slice. White: planned in Phase 1.

Step	What the tool does	Example (not measured yet)
1. Give the page	A person pastes a URL of a registered site and types the target keyword.	/blog/annapurna-base-camp-trek, keyword "Annapurna base camp trek".
2. Fetch	Fetches that one page through the safe fetcher (Part 6).	1 request, after the robots.txt check.
3. Extract	Reads the SEO parts: title, description, headings, images, links, structured data.	The H1, the images and their alt text, the BlogPosting data.
4. Audit	Runs SEO rules for the page type. Each finding has evidence and a source.	A title wider than about 600 px can be cut (rule of thumb, R3 §1).
5. Suggest	DeepSeek writes fixes from the page facts only. A missing fact becomes [ADD: ...].	A shorter title that keeps the keyword.
6. Review	A person accepts, edits or rejects each suggestion, and copies it into the site editor.	The person saves the page in MyPipit. The tool never saves.
7. Check again	Fetches the page again and compares the two audits.	A fixed finding is gone; a new one shows.

Later: the whole site

When the loop works for one page, the tool scales up: more page types (Phase 2), the whole site with crawl, schedules and Search Console (Phase 3), the browser extension (Phase 4), extendmy.life (Phase 5), and measured results (Phase 6). The tool is general: MyPipit and extendmy.life are test sites, so a later client needs no new code.

Rules that never change (CLAUDE.md)

- The tool only reads client sites. It never publishes. A person approves and saves each change.
- AI only reads, sorts and writes text. Code does all counts and scores.
- A suggestion uses only facts from its page. Each other fact becomes [ADD: ...].
- Health pages need an expert review. German pages need a native reviewer.

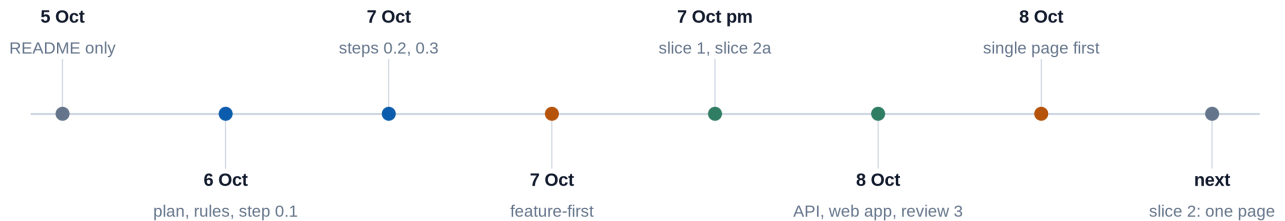
PART 2

Where we are

What is done, how we work, and what changed on the way.

The repository started on 5 October 2026 with an empty README. On 8 October it has a safe fetcher, a sitemap reader, a database with two tables, an API, a web app and 175 Python tests. The work of 8 October is not committed yet.

From an empty repository to a single-page plan in four days.



51 commits: 1 on 5 Oct, 11 on 6 Oct, 39 on 7 Oct. The 8 Oct work is not committed yet.

Figure 2. Most commits happened on 7 October. The work of 8 October is in the working tree.

The numbers

What	Count	How it was counted
Commits	51 (head 877c666)	<code>git rev-list --count HEAD</code>
Python source files	29 files, 1929 lines	<code>apps/api/src, wc -l</code>
Python tests	175 (162 unit, 13 integration)	<code>pytest --collect-only, -m unit, -m integration</code>
Web app	51 files, 2729 lines, 11 tests	<code>apps/web/src, wc -l; vitest</code>
API routes	5	<code>/health, 2 sites routes, 2 inventory routes</code>
Database tables	2 (tenants, sites)	<code>op.create_table</code> in the 1 migration
Import contracts	5	<code>pyproject.toml</code>
Packages in uv.lock	79	<code>uv.lock</code>
Session notes	8	<code>docs/notes/</code>

Status of each step

Step	Status	Result
0.1 to 0.3	Done	Machine, skeleton, Python base: lint, types, tests, import rules, Settings.
Slice 1	Done	Safe fetcher, 2 security reviews, sitemap reader. The list is now the page picker.
Slice 2a	Done	PostgreSQL 18 in Docker, Alembic, tenants and sites, make seed.
API and web app	Done	5 routes, RFC 9457 errors, request trace, 3rd security review, make dev.
Replan	Done	Single page first. PLAN.md, CLAUDE.md, README and both PDFs changed.
Slice 2	Next	Fetch one blog post, extract its SEO content, then its screen.
Slices 3 to 5	Planned	Audit with evidence, DeepSeek suggestions, test on 5 posts.

How we work

Each small part goes through five stages. CLAUDE.md sets this order.

1. **Research.** Read only the source that the task names. Write the question.
2. **Experiment.** A short throwaway script in experiments/ answers the question.
3. **Implement.** Write the tests first. Make sure that they fail. Then write the code.
4. **Understand.** Walk the owner through the code. Write a note in docs/notes/.
5. **Review the plan.** If a decision changed, update PLAN.md and add a change-log line.

In each slice, the backend comes first. Its screen comes after the backend works.

Two changes of direction

7 October, feature-first. The first plan put all setup first. You asked why we make folders for features that do not exist. Now each slice builds one feature from end to end, and each setup step starts when a slice needs it.

8 October, single page first. The plan started with the whole site: every page, crawled and audited on a schedule. You asked why we check the whole site when the SEO work is on one page. Now Phase 1 is one MyPipit blog post from end to end. The whole site is Phase 3. The code that was built stays: the safe fetcher is needed for one page too, and the sitemap list helps to pick the page.

NOTE

Nothing that was built is lost. The sitemap reader becomes the page picker, and in Phase 3 it is the start of the whole-site inventory.

PART 3

Words you need

The terms that the rest of this document uses.

Term	Meaning in this project
Page loop	The 7 steps for one page: give, fetch, extract, audit, suggest, review, check again.
Slice	One small feature from end to end, in one pull request. Phase 1 has 5 slices.
Spike	A throwaway experiment in experiments/. Production code never imports it.
robots.txt	A file on each site. It says which URLs a crawler may read (RFC 9309).
Sitemap	An XML file that lists the URLs of a site (sitemaps.org). An index lists other sitemaps.
Page type	The kind of page, for example blog_post or listing. It comes from URL patterns.
SSRF	Server-side request forgery. An attacker makes our server request an internal address, for example the cloud metadata address 169.254.169.254.
DNS rebinding	A trick: a name points to a public address at the check, and to a private address at the connection.
Gzip bomb	A small compressed file that unpacks to a very large size and fills the memory.
Rate limit	A minimum time between two requests to the same site. Here it is 1 second.
Tenant	A client company. Every row in the database belongs to one tenant.
Snapshot	One saved fetch of a page: its HTML and the SEO parts that the tool read from it.
Finding	One SEO problem on a page, with evidence (where, what was found, what is expected) and a source.
Suggestion	A fix that the AI writes. A person accepts, edits or rejects it. The tool never publishes it.
Migration	A versioned script that changes the database tables (Alembic).
uuidv7()	A PostgreSQL 18 function that makes IDs that sort by time.
Import contract	A rule about which code may import which code. import-linter checks it.
Fixture	Saved test data, for example the real MyPipit sitemap, so tests need no network.
testcontainers	A test tool that starts a real PostgreSQL in Docker for the tests.
7-day delay	uv does not install a package version that is less than 7 days old.
ASD-STE100	Simplified Technical English. All documents in this project use it.

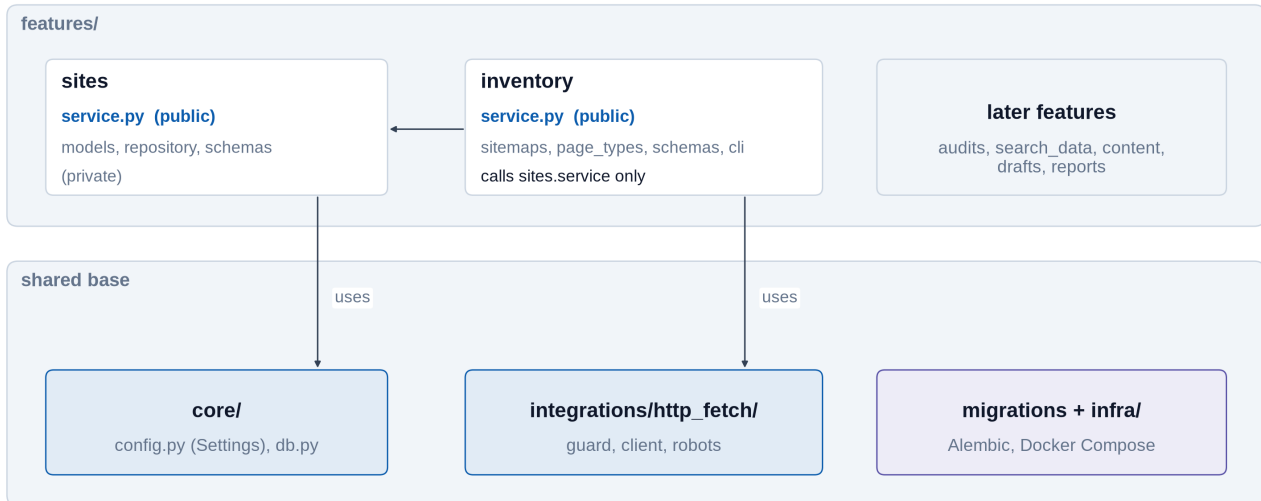
PART 4

How the code is organised

One folder for each job, and rules that keep the folders apart.

The code is a modular monolith. It is one program, but each job has its own folder (a feature). This keeps the program simple to run and the parts simple to change.

Features call each other only through service.py. The shared base never calls a feature.



3 import contracts, checked by mise run check: the shared base never imports a feature; sites and inventory keep their internals private.

Figure 3. inventory gets the site data only from sites.service. If it imported sites.models, mise run check would fail.

The five import contracts

Contract	What it stops	Why
core and integrations never import a feature	core/ or integrations/ uses features/	The shared base stays small, with no hidden links.
sites: internals are private	Another feature uses sites.models, repository or schemas	You can change a sites table and no other feature breaks.
inventory: internals are private	Another feature uses sitemaps, page_types or schemas	Same reason. Other features call inventory.service.
sites, inventory: only main.py mounts the router	Another module imports a feature's api.py	main.py is the one place that builds the HTTP app.

RULE

A new feature adds two contracts: one for its internals and one for its router. The template is at the end of pyproject.toml. One general rule cannot say "only its own feature", so each feature needs its own.

The file map

File	Lines	Responsible for
main.py	68	Builds the API app, mounts the routers, one SafeFetcher per app
core/api.py	148	RFC 9457 errors, request ID, Host check, request dependencies
core/config.py	53	Settings: reads 8 values from the environment, stops if one is wrong
core/db.py	48	Database base: name rules, uuidv7() keys, timestampz columns, engine
integrations/http_fetch/guard.py	197	SSRF guard, pinned connection, total time limit
integrations/http_fetch/client.py	204	Guarded client, size limit, own gzip decoder, rate limiter

File	Lines	Responsible for
integrations/http_fetch/robots.py	285	robots.txt rules, SafeFetcher, redirects, trace events
integrations/http_fetch/trace.py	69	Record of each request: URL, status, size, time
features/sites/service.py	79	register_site, get_site, list_sites, load_site_spec
features/sites/api.py, cli.py	57	GET /api/v1/sites routes; make seed
features/inventory/sitemaps.py	65	Safe sitemap XML reader, 50 MB and 50,000 entries
features/inventory/page_types.py	39	URL pattern to page type, and which rule matched
features/inventory/service.py	211	discover_pages, run_discovery, find_page_type
features/inventory/api.py, cli.py	81	discovery and page-type routes; inventory:discover

The source has 29 files and 1929 lines. The table lists the files with logic; the other files are short package files. migrations/ and infra/ are outside the Python package.

PART 5

The main trace: inventory:discover on MyPipit

One command, followed step by step through the code.

This part follows one real run. It touches almost every part that we built. The command is below. It ends when the terminal prints 58 pages.

```
$ mise run inventory:discover -- infra/sites/mypipit.toml
Site: https://marketplace.mypipit.com
Sitemaps read: 1
Pages: 58
  blog_post  30  listing  17  blog_category  5  static  3
  listing_index  1  blog_index  1  seller  1
Problems: 0
```

One run of inventory:discover. 2 requests reach MyPipit; 58 pages come back.

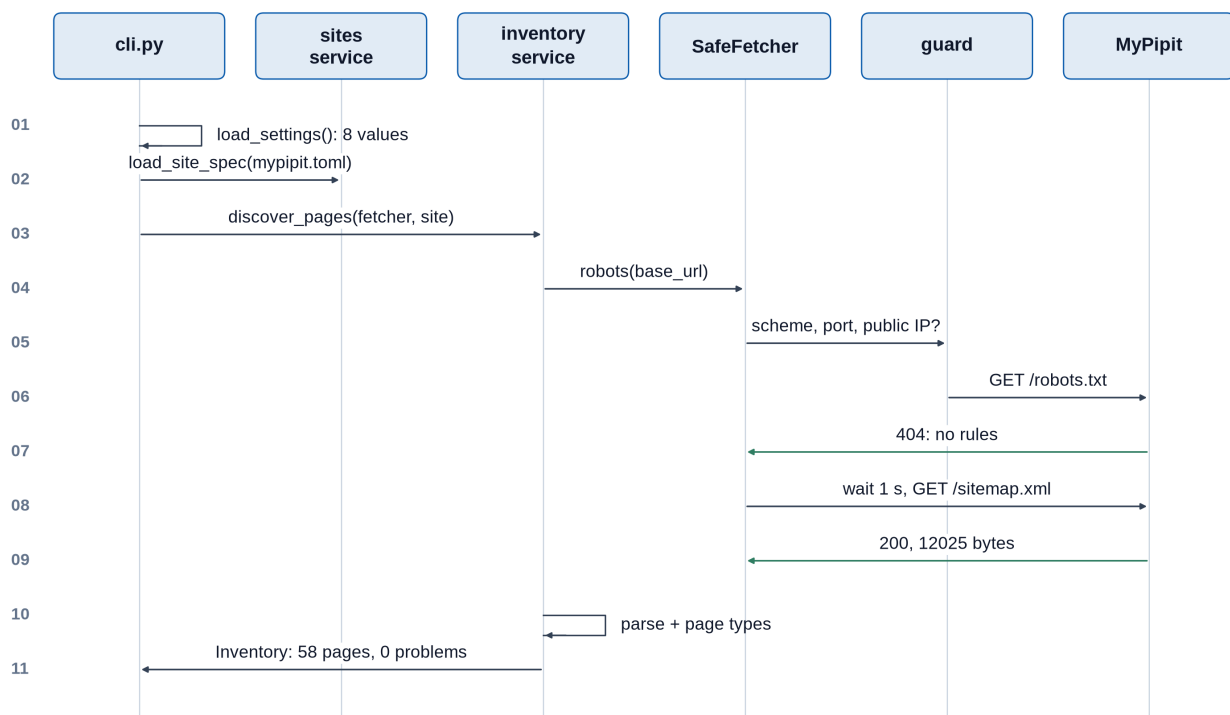


Figure 4. Steps 1 to 3 prepare. Steps 4 to 9 are the network part: only 2 requests reach MyPipit. Steps 10 and 11 turn bytes into pages.

Steps 1 and 2. Load the settings and the site file

cli.py reads the 8 settings. If one is missing, it prints a clear message and stops with exit code 2. Then it reads the site file. The file is data, not code.

```
base_url = "https://marketplace.mypipit.com"
language = "en"
page_types = [
    { pattern = "/blog", page_type = "blog_index" },
    { pattern = "/blog/category/*", page_type = "blog_category" },
    { pattern = "/blog/*", page_type = "blog_post" }, # first match wins
    ...
]
```

Step 3. Make the fetcher and start discovery

cli.py turns 5 settings into a FetchConfig and opens a SafeFetcher. Then it calls discover_pages. This function does not know the site file format. It needs only 3 fields: base_url, language and page_types.

Steps 4 to 7. Read robots.txt

discover_pages asks the fetcher for the robots.txt rules. The fetcher checks the address first (Part 6). Then it requests /robots.txt. MyPipit answers 404.

```
if status is None or status >= 500 or status == 429 or 300 <= status < 400:
    return DISALLOW_ALL, True      # unreachable: block everything
if status in (401, 403):
    return DISALLOW_ALL, False     # stricter than the RFC, on purpose
if status >= 400:
    return "", False               # 404: no file, so no rules
```

NOTE

404 means "no rules", so every page is allowed (RFC 9309). A 5xx answer is the opposite: the site has a problem, so the tool reads nothing.

Steps 8 and 9. Read the sitemap

robots.txt has no Sitemap: line, so discover_pages uses /sitemap.xml. The fetcher checks robots.txt, waits 1 second for the rate limit, and downloads 12025 bytes.

Step 10. Parse and sort

parse_sitemap reads the XML with external entities off and recover=False, so broken XML is an error. Each URL gets a page type from the first pattern that matches. * is exactly one path part.

```
for rule in rules:
    if _compile(rule.pattern).fullmatch(path):
        return rule.page_type
return OTHER
```

Step 11. Return the inventory

discover_pages returns 58 pages and 0 problems. A problem with one sitemap does not stop the others. It goes into the problems list, because people must see it.

The same run through the API

POST /api/v1/sites/{id}/discoveries runs the same steps for a stored site. It also records each request in a trace. On a second run in 24 hours, robots.txt comes from the cache, so only 1 request reaches MyPipit. In the single-page plan, this list is the page picker.

Steps and their files

Step	What happens	File
1	Read 8 settings	core/config.py
2	Read the site file	features/sites/service.py
3	FetchConfig, SafeFetcher, discover_pages	features/inventory/cli.py
4	Ask for robots.txt rules	features/inventory/service.py
5	Check scheme, port, public address	integrations/http_fetch/guard.py
6, 7	GET /robots.txt, 404 means no rules	integrations/http_fetch/robots.py
8, 9	Rate limit, GET /sitemap.xml with limits	integrations/http_fetch/client.py
10	Parse XML, page types	features/inventory/sitemaps.py, page_types.py
11	Inventory with pages and problems	features/inventory/service.py

PART 6

The safe fetcher

Why the internet path has seven gates, and how two reviews found its weak points.

The tool reads URLs that other people choose: site owners, sitemaps and redirects. Any of them can point to an internal address. So every outbound request in the project goes through one package: `integrations/http_fetch`. `CLAUDE.md` requires this.

Every hop of every fetch passes all seven gates. Any gate can stop it.

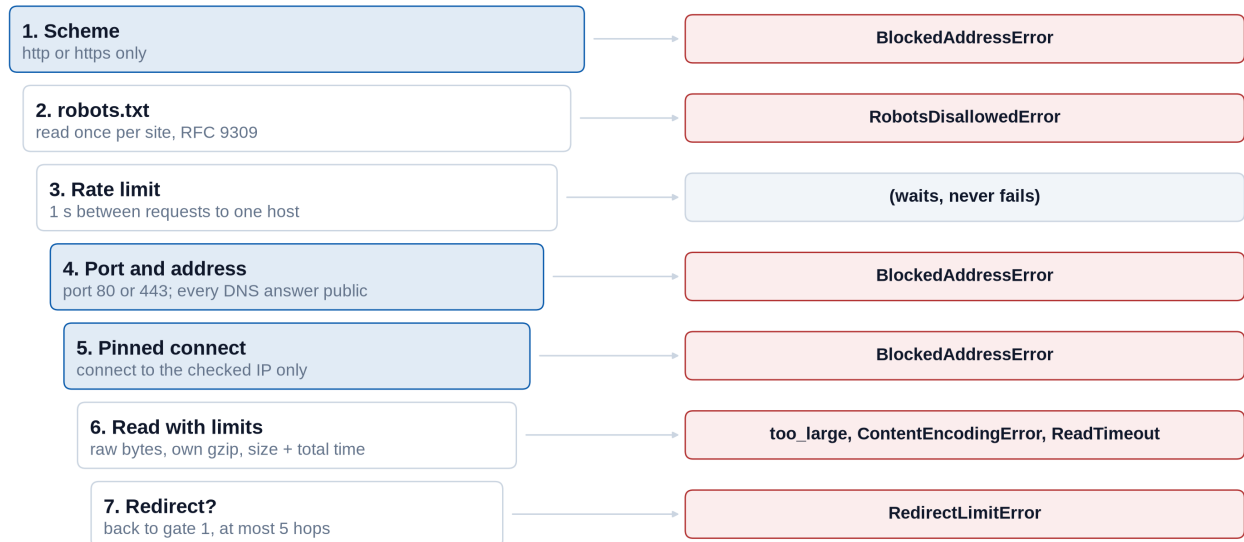


Figure 5. The gates run again for each redirect. A redirect to 10.0.0.5 stops at gate 4, before any byte leaves the machine.

What each gate does

Gate	How	Why
1 Scheme	<code>require_web_scheme</code> on every hop	<code>ftp:</code> , <code>file:</code> and <code>javascript:</code> are not web pages.
2 robots.txt	Read once per site (scheme, host, port); kept 24 hours	Product rule: obey robots.txt. RFC 9309 limits the cache to 24 hours.
3 Rate limit	<code>HostRateLimiter</code> reserves a time slot per host, under a lock	Politeness. <code>S.example</code> , <code>s.example:443</code> and <code>a@s.example</code> are one host.
4 Port and address	<code>guard_request</code> hook: port 80 or 443; all DNS answers public	One private answer blocks the host. Answers are never cached.
5 Pinned connect	<code>PublicOnlyBackend</code> connects to the checked IP	Stops DNS rebinding between the check and the connection.
6 Read with limits	Raw bytes, own gzip decoder, size limit, total deadline	Stops gzip bombs and servers that send 1 byte at a time.
7 Redirect	<code>SafeFetcher</code> follows redirects itself; maximum 5	<code>htpx</code> would read every redirect body without a limit.

The constants

Name	Value	Source
<code>ALLOWED_SCHEMES</code>	<code>http</code> , <code>https</code>	<code>guard.py</code>
<code>ALLOWED_PORTS</code>	80, 443	<code>guard.py</code> ; owner decision on 7 October
<code>ROBOTS_MAX_BYTES</code>	512 000	RFC 9309 section 2.5: read at least 500 KiB
<code>ROBOTS_MAX_AGE_S</code>	24 hours	RFC 9309 section 2.4

Name	Value	Source
MAX_REDIRECTS	5	RFC 9309 section 2.3.1.2 asks for at least 5
SITEMAP_MAX_BYTES	50 MB	sitemaps.org limit, uncompressed
FETCH_MIN_DELAY_S	1.0 (setting)	.env; R1 section 3.5

Where the code came from

Most of the gates came from your older project, SEOAdvisor (base.py and fetcher.py). We took only what slice 1 needs, and we changed two things. First, the production code has no test hook any more; the tests replace the DNS lookup. Second, an SSRF block stays a clear `BlockedAddressError`. The old code turned it into "robots.txt could not be read".

Two security reviews

CLAUDE.md asks for a security review before a pull request that changes fetching. A separate review agent read the code and the `httpx` and `httpcore` source. The first review found 9 problems. Most of them came with the old code.

#	Level	Problem	Fix
1	High	<code>httpx</code> unpacked gzip before the size check, and accepted "gzip, gzip"	Raw bytes, own decoder with a limit, one gzip only
2	High	<code>httpx</code> read every redirect body, up to 20 hops	No auto redirects; body never read
3	Med-high	The time limit missed connect, TLS and headers	One total deadline on every read
4	Medium	Redirects skipped robots.txt and the rate limit	All gates on every hop
5	Medium	Three spellings of one host were three rate-limit keys	One normalised key
6	Low-med	NAT64 forms of private IPv4 passed	Check the IPv4 inside
7	Low	A rebinding block looked like unreachable robots.txt	Backend raises <code>BlockedAddressError</code>
8	Low	Ports, secrets in messages, large robots.txt, cache age	Ports 80/443, clean messages, RFC rules
9	Tests	Tests did not use the real guard path	Fake DNS and the real hook in each test

TRAP

Before the fix, a response of 173 bytes made the memory grow by 18 MB with a 1 MB limit. A third gzip layer reaches gigabytes. After the fix, the stacked form is refused, and a 200 MB bomb uses no extra memory.

The second review confirmed all 9 fixes. It found 4 new small problems: a `KeyError` on a redirect to ftp:, a time limit for each hop and not for the whole request, a robots.txt 304 that meant "allow all", and a size limit below 1 byte. We fixed all 4 with tests first.

The tests that prove it

- 23 address cases, for example 172.16.0.1, fc00::1 and 64:ff9b::7f00:1. Each must be blocked.
- A redirect to a private address is never sent. The test checks the list of sent requests.
- A real client with fake DNS: public at the check, 127.0.0.1 at the connection. Result: a clear block.
- The integrations folder has 102 tests in total. They use no network.

NOTE

One limit stays. Python's DNS lookup has no timeout of its own. The system resolver stops after a few seconds. The note for slice 1 records this.

The request trace

On 8 October the fetcher learned to record each request. A caller opens `trace_scope()`, and every hop inside it adds one event: kind, URL, outcome, status, bytes, wait and time. The API returns these events, so people can see exactly what the tool sent.

- The URL has no query and no user info, because these can hold secrets.

- The trace keeps no bodies and no headers. An error event keeps only the error type.
- Recording never changes a check. Each except block records the event and raises the error again.
- A cached robots.txt shows once per trace, so the trace says where the rules came from.

The third review

The API made the fetch reachable over HTTP, so a third review read the new code. It found no high problem, 2 medium and 4 low. All 6 are fixed, with tests first.

#	Level	Problem	Fix
1	Medium	A hostile sitemap could name thousands of other hosts	Only sitemaps on the site's own host
2	Medium	DNS rebinding could reach the local API	Host check: 127.0.0.1 and localhost only
3	Low	A database connection stayed open during the network part	Close the session before the fetch
4	Low	A malformed URL in the URL tester gave 500	422 with a clear message
5	Low	A javascript: URL could reach the pages list	Only http and https page URLs
6	Low	A network error showed system details	Only the error type

PART 7

The inventory

How the tool turns a sitemap into a list of typed pages.

The inventory answers one question: which pages does this site have? In the single-page plan, this list is the page picker. In Phase 3 it becomes the start of the whole-site work.

What the experiment found

Before we wrote the real code, a spike read MyPipit one time with 2 requests. The results are in `experiments/2026-10-07-mypipit-sitemap/README.md`.

Finding	Value
robots.txt	Does not exist (404). So there is no Sitemap: line.
Sitemap	One file, /sitemap.xml, 12025 bytes, not an index.
URLs	58, all on marketplace.mypipit.com, no query strings, no hreflang.
lastmod	51 of 58 URLs have a date.
Language	All English.

The MyPipit page types

Pattern	Page type	Pages
/blog	blog_index	1
/blog/category/*	blog_category	5
/blog/*	blog_post	30
/marketplace	listing_index	1
/marketplace/*	listing	17
/pages/*	static	3
/sellers/*	seller	1
(no match)	other	0

NOTE

/marketplace/* mixes treks, tour packages and guide profiles. The URL alone cannot separate them. Phase 2 can use the page content for a finer type.

Rules for sitemaps

- A sitemap may list only URLs on its own host (sitemaps.org). A URL on another host is reported.
- An index may not list another index (sitemaps.org). A nested index is reported and not followed.
- A file may have at most 50,000 entries and 50 MB (sitemaps.org).
- Our rule: the tool reads only sitemaps on the site's own host, so one run reaches one host.

Problems are data

A problem with one sitemap does not stop the run. It goes into a list, because each problem is an SEO finding that people must see. The service catches only named error types, never all errors.

Problem kind	When
sitemap_blocked	robots.txt or the address guard does not allow the sitemap
sitemap_unreadable	HTTP error, too large, or a network error

Problem kind	When
sitemap_invalid	Not valid XML, or the root is not urlset or sitemapindex
nested_sitemap_index	An index inside an index
sitemap_on_other_host	A sitemap on another host: reported, never requested
url_on_other_host	A URL on a host that is not the site's host
url_not_http	A page URL that is not http or https, or that cannot be read
duplicate_url	The same URL a second time

The inventory folder has 37 tests. One of them uses the real MyPipit sitemap as a saved file, so it checks all 58 pages and the 7 page type counts without the network.

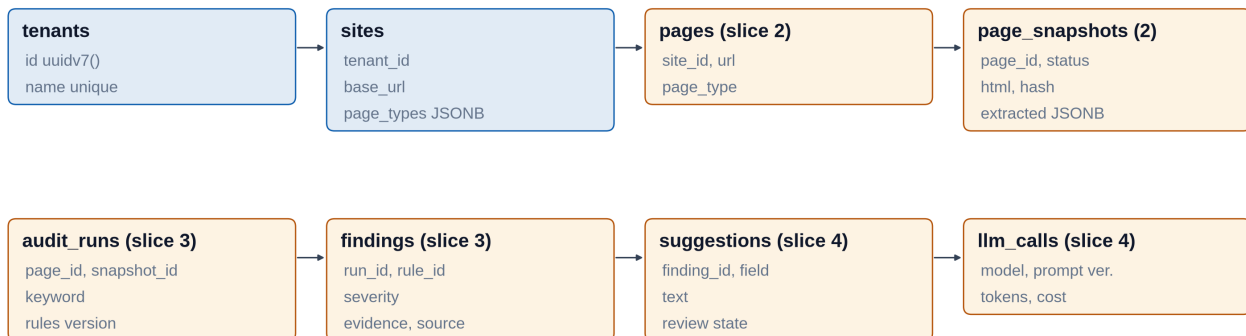
PART 8

The database

PostgreSQL 18 in Docker, two tables, and migrations that we read by hand.

Slice 2a made the base and the tables for tenants and sites. The single-page plan adds a table for each step of the page loop, one slice at a time.

Built tables (blue) and the tables of the page loop (amber), one slice at a time.



pages holds only pages that a person analyses, not every sitemap URL.

Every row has tenant_id and site_id. Times are timestampz (UTC).

Figure 6. Built tables (blue) and the tables of slices 2 to 4. Every row has tenant_id and site_id.

The local database

Item	Setting	Why
Image	pgvector/pgvector:0.8.7-pg18-trixie	Full tag pinned. The tests use the same image.
Data volume	/var/lib/postgresql	PostgreSQL 18 rule. The old path breaks start-up.
Port	127.0.0.1:5432	This machine only.
Health check	pg_isready every 2 s	mise run db:up waits until the database is ready.
Values	from .env	No password in the Compose file.

The experiment showed PostgreSQL 18.6, version-7 IDs from uuidv7(), and the time zone UTC.

Design points

- IDs come from PostgreSQL: `server_default=text("uuidv7()")`.
- Times are timestampz. The database sets them.
- `register_site` writes only if a value changed. A second run writes nothing.
- An update sets `updated_at` to `clock_timestamp()`, not `now()`. `now()` is the start of the transaction.
- The site page types are stored as JSONB, so the rules stay data.

The first migration

Alembic made the first migration automatically. We read it and changed two things before the commit. The unique rule now has a name with both columns: `uq_sites_tenant_id_base_url`. The extra index on `sites.tenant_id` is gone, because the unique rule already starts with `tenant_id`.

RULE

Read and edit every generated migration (R4 section 13.2). Never edit a migration after it is merged. Write a new one.

How the tests use the database

1. testcontainers starts one PostgreSQL 18 container for the test session.
 2. Alembic upgrades it to the latest migration one time. This also tests the migrations.
 3. Each test runs in a transaction. Commits become savepoints. The transaction rolls back after the test.
 4. A test runs `alembic check`. It fails if a model changed without a migration.
- 6 tests use the database. The full test run takes about 6 to 12 seconds.

The next tables

Slice	Table	What a row is
2	pages	A page that a person analyses: site, URL, page type. Not every sitemap URL.
2	page_snapshots	One fetch: HTML with a size limit, hash, final URL, status, extracted fields.
3	audit_runs	One audit: page, snapshot, keyword, rules version.
3	findings	One problem: rule, severity, evidence (where, found, expected), source.
4	suggestions	One AI fix: field, text, review state (accepted, edited, rejected).
4	llm_calls	One AI call: model, prompt version, tokens, cost.

~~Part 2b: save all 58 sitemap pages~~ moved to Phase 3. Only analysed pages are saved now.

PART 9

Tools and checks

What each tool prevents, and what mise run check does.

Every tool here prevents one kind of problem. Together they find most errors before a person sees the code. This is important when AI writes code.

mise: one place for versions and commands

mise.toml pins 6 tools: Python 3.14.8, uv 0.12.23, Node 24.21.0, pnpm 12.9.1, gitleaks 8.30.1 and prek 0.5.5. It also defines its tasks. A Makefile gives short names: make dev runs mise run dev. The ones that work now are below.

Command	What it does
make dev	Starts the database, the API (port 8000) and the web app (port 5173)
mise run setup	Installs the tools and packages, makes .env, saves the sites
mise run check	lint, typecheck and tests (Python and web), in parallel
mise run audit	pip-audit on uv.lock (all groups) and pnpm audit
mise run db:seed	Saves the sites in infra/sites/*.toml
mise run gen-client	Writes openapi.json and the typed client in packages/api-client
mise run docs:pdf	Builds the requirements PDF (docs packages)
mise run db:up / db:down	Starts or stops PostgreSQL
mise run db:migrate	Applies the migrations
mise run inventory:discover -- <site file>	Prints the pages of a site

What mise run check runs

Tool	Version	Prevents
ruff check	0.16.9	Common errors and unsafe code. 13 rule sets, for example S (security).
ruff format	0.16.9	Differences in code format
lint-imports	2.15	Features that use each other's internals (5 contracts)
mypy --strict	2.3.1	Type errors, with the Pydantic plugin
pytest	9.1.1	Behaviour changes. 175 tests, strict mode, 4 markers
web checks		oxlint, Prettier, tsc and Vitest (11 tests) for the web app

The 7-day delay

pyproject.toml sets exclude-newer to 7 days. uv does not install a version that is less than 7 days old. New releases sometimes contain attacks or bugs that people find in the first days. Example: the plan named ruff 0.16.10 and mypy 2.4.0. Both were 6 days old, so uv installed ruff 0.16.9 and mypy 2.3.1.

Packages and licences

Package	Version	Licence	Use
httpx, httpcore	0.28.1, 1.0.9	BSD-3-Clause	HTTP client
protego	0.7.0	BSD-3-Clause	robots.txt rules
lxml	6.1.3	BSD-3-Clause	Sitemap XML
pydantic, pydantic-settings	2.13.5, 2.15.0	MIT	Data shapes, Settings
SQLAlchemy	2.1.1	MIT	Database code

Package	Version	Licence	Use
psycopg (binary)	3.3.6	LGPL-3.0	PostgreSQL driver, used unchanged
Alembic	1.20.0	MIT	Migrations
testcontainers	4.15.0	Apache-2.0	Tests only
pytest, ruff, mypy	9.1.1, 0.16.9, 2.3.1	MIT	Checks
import-linter, pip-audit	2.15, 2.10.1	BSD-2, Apache-2.0	Checks
fastapi, uvicorn[standard]	0.142.2, 0.54.0	MIT, BSD-3-Clause	API
playwright	1.63.0	Apache-2.0	Docs only: prints the requirements PDF
@hpcjs/wasm-graphviz	1.29.2	Apache-2.0	Docs only: draws the PDF diagrams
Web app packages	32 in package.json	MIT, ISC, Apache-2.0, OFL	Web app; 7-day delay, no install scripts

You approved each package before we added it. mise run audit finds no known vulnerabilities.

Settings

core/config.py reads 8 values from the environment. mise loads .env into the environment. Each value is required and has no default, so a missing value cannot hide.

```
$ (no DATABASE_URL, LOG_LEVEL=loud)
SettingsError: Settings are missing or invalid: LOG_LEVEL (invalid),
DATABASE_URL (missing). Set them in the environment or in .env (see .env.example).
```

RULE

The message names each problem but never shows the value. Pydantic's own message repeats the value, and the value can be a secret.

PART 10

Rules and decisions

The working agreements and the decisions that shape the code.

Rules that you added to CLAUDE.md

Rule	What it means
ASD-STE100	All documents and explanations use Simplified Technical English.
No hidden errors	No bare except, no except Exception: pass, no default value in place of a failure.
Commits	Only when you ask. One short line, 50 characters or less. No mention of Claude or AI.
Branches	Claude never merges a branch and never deletes a branch.
Folders	Make a folder only with its first file. No empty placeholders.

Decisions that you made

Date	Decision
6 Oct	PLAN.md and CLAUDE.md stay at the repository root.
7 Oct	Frontend: React with TanStack Router, Query and Table, Tailwind CSS and shadcn/ui.
7 Oct	CONTRIBUTING.md and SECURITY.md in step 0.2. Branch flow: branch from dev, pull request into dev.
7 Oct	Feature-first: Phase 1 in 5 slices. Setup steps start when a slice needs them.
7 Oct	First goal: MyPipit pages and SEO problems, with evidence.
7 Oct	Reuse the SEOAdvisor fetcher, but only what slice 1 needs.
7 Oct	lxml in place of defusedxml. MyPipit page types as listed in Part 7.
7 Oct	Fix all 9 security findings before the commit. Ports 80 and 443 only.
7 Oct	Packages for the database: SQLAlchemy, psycopy, Alembic, testcontainers.
8 Oct	Backend and frontend together; make dev. API packages fastapi, uvicorn; jsdom. Fix all 6 review findings.
8 Oct	Single page first: paste a URL and a keyword, blog posts first, audit then AI (DeepSeek).
8 Oct	Sitemap list = page picker. Backend first, then its screen. Docs packages for the PDF.

Decisions that Claude made for you

These are in the notes. Tell Claude if you disagree with one.

- No pgvector extension yet. No table needs vectors.
- mise run check needs Docker, because the tests use a real database.
- The site data (SiteSpec, page types) belongs to the sites feature, as CLAUDE.md says.
- Rules live in code, each with a source, not in a rules table. The rules table left the plan.
- The global uv in ~/.local/bin was updated to 0.12.23 (you chose this), so PATH order does not matter.

Git

All work is on dev and main at the same commit. Claude makes the commits on dev and moves main forward, with no merge commit. Before each push, gitleaks scanned the files and mise run check passed on the last commit. The work of 8 October is not committed yet; it waits for your request.

TRAP

On 7 October, a commit script put a file into the wrong commit before the push. Claude made the 10 commits again. The script now stops at the first error.

PART 11

Open items and next steps

What is not done, what is only partly done, and what comes next.

Open items

Item	Status
Slice 1 walk-through and plan review	Waits for you. Read the slice 1 and slice 2 notes first.
Requirements PDF	Rebuilt on 8 October for the single-page plan (docs packages approved).
lxml-stubs	Waits for your decision.
Retry-After and back-off for 429 and 503	Not built. The crawler in Phase 3 needs it.
DeepSeek key and data terms	You give the key before slice 4. Check its data terms then.
Total limit for one discovery	No limit on sitemaps or pages per run yet. Needs a Settings value.
htpx2 warning in the tests	Starlette asks for httpx2. A new package: your decision.
DNS lookup timeout	Known limit. The system resolver sets it.
Git hooks and CI	Not built. Step 0.7 and 0.8, before the first pull request.
ADRs	None yet. Each ADR comes when its decision is first used.
security-reviewer subagent	Does not exist. Reviews used a general agent with a security prompt.
Open questions in PLAN.md	9, for example Search Console access and the DeepSeek key.
Card in Emitii	No card matches this work. Your 7 October hours have no card.

Next steps

Slice	What you will see	New setup
2	Paste a blog post URL; see its title, headings, images, links, data	pages, page_snapshots
3	SEO findings for that page, each with evidence and a source	audit_runs, findings
4	DeepSeek suggestions that you accept, edit or reject	AI layer, suggestions, llm_calls
5	The loop on 5 blog posts with the SEO team	None

How to start the next session

1. Run `mise run db:up`.
2. Start Claude Code in the project folder.
3. Type: "Read PLAN.md and the latest note in docs/notes/. Start slice 2 (backend first)."

When something fails

Symptom	Check first
mise errors that say "not trusted"	Run <code>mise trust</code> in the project folder.
SettingsError at start-up	Compare <code>.env</code> with <code>.env.example</code> . The message names the missing key.
mise run check fails in the tests	Is Docker running? The database tests need it.
RobotsDisallowedError	Read the message: "disallowed" or "could not be read".
BlockedAddressError	The URL is not http(s), not port 80 or 443, or not public.